



Министерство науки и высшего образования Российской Федерации
Федеральное государственное автономное образовательное учреждение
высшего образования
«Московский государственный технический университет
имени Н.Э. Баумана
(национальный исследовательский университет)»
(МГТУ им. Н.Э. Баумана)

ФАКУЛЬТЕТ «Информатика и системы управления» (ИУ)

КАФЕДРА «Информационная безопасность» (ИУ8)

ТЕОРИЯ ИГР И ИССЛЕДОВАНИЕ ОПЕРАЦИЙ

Лабораторная работа № 6

на тему:

«Информационное противоборство в социальных сетях»

Вариант 13

Студент:

ИУ8-105

(Группа)

Б. Е. Шекунов

(И. О. Фамилия)

Преподаватель:

Н. С. Коннова

(И. О. Фамилия)

Москва, 2026 г.

Цель работы:

Изучить теоретико-игровую модель информационного противоборства в социальных сетях. Промоделировать процесс изменения мнений агентов без управляющего воздействия и с информационным управлением, а также определить итоговое мнение агентов и победителя информационного противоборства.

Постановка задачи:

Рассматривается социальная сеть, состоящая из 10 агентов. Каждый агент имеет некоторое начальное мнение, которое изменяется под влиянием мнений остальных агентов.

Изменение мнений задаётся законом:

$$x(t) = Ax(t - 1),$$

где: $x(t)$ – вектор мнений агентов на шаге t ,

A – матрица доверия, $x(0)$ – начальный вектор мнений агентов.

Матрица доверия должна быть стохастической по строкам, то есть все её элементы неотрицательны, а сумма элементов каждой строки равна 1:

$$a_{ij} \geq 0,$$
$$\sum_{j=1}^n a_{ij} = 1.$$

В работе необходимо:

1. Сгенерировать стохастическую матрицу доверия для 10 агентов.
2. Задать начальный вектор мнений и определить итоговое мнение без информационного управления.
3. Выбрать непересекающиеся множества агентов влияния первого и второго игроков. Назначить агентам влияния управляющие значения мнений игроков.
4. Выполнить моделирование с информационным управлением.
5. Определить итоговое мнение и победителя.

Исходные данные:

В результате генерации получена следующая матрица доверия:

1. Сгенерированная матрица доверия

A =

0.037	0.132	0.097	0.077	0.073	0.163	0.187	0.037	0.135	0.062
0.164	0.156	0.108	0.127	0.087	0.140	0.076	0.057	0.047	0.038
0.132	0.108	0.167	0.003	0.113	0.092	0.049	0.150	0.110	0.076
0.035	0.146	0.133	0.101	0.058	0.158	0.094	0.072	0.150	0.053
0.147	0.066	0.055	0.148	0.048	0.104	0.122	0.040	0.154	0.116
0.128	0.077	0.087	0.102	0.097	0.139	0.119	0.086	0.001	0.164
0.108	0.068	0.104	0.020	0.188	0.206	0.012	0.075	0.152	0.066
0.100	0.074	0.137	0.169	0.157	0.110	0.028	0.167	0.004	0.053
0.060	0.143	0.104	0.020	0.003	0.129	0.105	0.128	0.120	0.189
0.143	0.029	0.146	0.122	0.099	0.103	0.085	0.143	0.036	0.093

Рисунок 1 – Сгенерированная матрица доверия

Сгенерированный начальный вектор мнений агентов без управляющего воздействия:

$$x(0) = (15; 17; 10; 3; 16; 13; 3; 9; 3; 16).$$

Ход работы:

Для начала проверим свойства сгенерированной матрицы доверия A.

Все элементы матрицы неотрицательны.

Сумма элементов каждой строки равна единице.

Следовательно, матрица A является стохастической по строкам и корректно описывает распределение доверия агентов.

Далее выполнялось последовательное вычисление:

$$x(t) = Ax(t - 1).$$

Процесс продолжался до тех пор, пока максимальное изменение компонент вектора мнений между соседними итерациями не стало меньше заданной точности:

$$\max_i |x_i(t) - x_i(t - 1)| < \varepsilon.$$

Точность остановки итерационного процесса: $\varepsilon = 10^{-6}$.

2. Моделирование без информационного управления

Начальные мнения $x(0)$: (15; 17; 10; 3; 16; 13; 3; 9; 3; 16)

Таблица сходимости:

t	$\max x(t) - x(t-1) $	$\min x(t)$	$\max x(t)$	разброс
1	8.79741387	9.483	11.797	2.314
2	1.45627344	10.578	11.048	0.470
3	0.28519530	10.762	10.847	0.085
4	0.05608389	10.789	10.799	0.010
5	0.00585366	10.793	10.795	0.001
6	0.00085207	10.794	10.794	0.000
7	0.00013327	10.794	10.794	0.000
8	0.00001943	10.794	10.794	0.000
9	0.00000330	10.794	10.794	0.000
10	0.00000057	10.794	10.794	0.000

Рисунок 2 – Таблица сходимости без информационного управления

После 10 итераций мнения всех агентов практически совпали:

$$x(10) = (10.794; 10.794; \dots ; 10.794; 10.794).$$

Последнее значение изменения:

$$\max_i |x_i(10) - x_i(9)| = 0.00000057.$$

Таким образом, без информационного управления социальная сеть приходит к единому итоговому мнению:

$$X \approx 10.793976.$$

При многократном умножении стохастической матрицы доверия степени A^k сходятся к матрице, строки которой практически одинаковы. Общая предельная строка имеет вид:

$$r = (0.106891; 0.099629; 0.113472; 0.088624; 0.093189; \\ 0.133759; 0.089602; 0.095231; 0.087161; 0.092441).$$

Тогда итоговое мнение можно проверить по формуле:

$$X = rx(0).$$

Для моделирования без управления получено:

$$X = 10.793976.$$

Это совпадает с результатом итерационного моделирования. Следовательно, расчёт итогового мнения выполнен корректно.

На следующем этапе моделируется информационное противоборство двух игроков.

Первый игрок стремится сместить итоговое мнение агентов к своему управляющему значению $u = 34$. Второй игрок стремится сместить итоговое мнение к своему управляющему значению $v = -44$.

Были выбраны следующие агенты влияния $I_1 = \{6, 8\}$ – агенты влияния первого игрока, $I_2 = \{9\}$ – агенты влияния второго игрока.

Множества агентов влияния не пересекаются: $I_1 \cap I_2 = \emptyset$.

Нейтральные агенты: $\{1, 2, 3, 4, 5, 7, 10\}$.

Их начальные мнения остаются неизменными.

С учётом информационного управления начальный вектор мнений принимает вид:

$$x(0) = (15; 17; 10; 3; 16; 34; 3; 34; -44; 16).$$

Здесь агентам 6 и 8 присвоено мнение первого игрока $u = 34$, а агенту 9 – мнение второго игрока $v = -44$.

Для вектора мнений с учётом управления снова применяется закон:

$$x(t) = Ax(t-1).$$

Таблица сходимости:

t	$\max x(t) - x(t-1) $	$\min x(t)$	$\max x(t)$	разброс
1	55.25814362	5.424	17.308	11.885
2	5.87301978	11.257	13.383	2.126
3	1.49509034	11.749	12.064	0.315
4	0.16145324	11.854	11.902	0.048
5	0.03333859	11.883	11.890	0.007
6	0.00367232	11.887	11.888	0.001
7	0.00085795	11.887	11.887	0.000
8	0.00010191	11.887	11.887	0.000
9	0.00002056	11.887	11.887	0.000
10	0.00000236	11.887	11.887	0.000
11	0.00000049	11.887	11.887	0.000

Рисунок 3 – Таблица сходимости с информационным управлением

После 11 итераций мнения агентов сошлись к одному значению:

$$x(11) = (11.887; 11.887; \dots ; 11.887; 11.887).$$

Последнее изменение мнений:

$$\max_i |x_i(11) - x_i(10)| = 0.00000049.$$

Проверка через предельную строку матрицы доверия:

$$X = rx(0) = 11.887155.$$

Итерационное моделирование дало:

$$X \approx 11.887151.$$

Различие связано только с округлением. Следовательно, результат моделирования согласуется с расчётом через предельную строку матрицы доверия.

Для определения победителя сравним расстояния итогового мнения до целей игроков.

Расстояние до цели первого игрока:

$$|X - u| = |11.887151 - 34| = 22.112849.$$

Расстояние до цели второго игрока:

$$|X - v| = |11.887151 - (-44)| = 55.887151.$$

Так как

$$22.112849 < 55.887151,$$

итоговое мнение оказалось ближе к цели первого игрока.

Следовательно, в данном информационном противоборстве победил первый игрок.

Без информационного управления итоговое мнение агентов составило:

$$X_0 \approx 10.793976.$$

С информационным управлением:

$$X_1 \approx 11.887151.$$

Изменение итогового мнения:

$$\Delta X = X_1 - X_0 = 11.887151 - 10.793976 = 1.093175.$$

Таким образом, информационное управление изменило итоговое мнение социальной сети примерно на 1.093175.

Несмотря на то, что второй игрок задал достаточно сильное отрицательное управляющее мнение -44 , влияние первого игрока оказалось сильнее с точки зрения итогового результата. Это объясняется тем, что у первого игрока было два агента влияния, а у второго только один агент.

Кроме того, веса предельной строки для агентов 6 и 8 равны:

$$r_6 = 0.133759,$$

$$r_8 = 0.095231,$$

Суммарная предельная влияние агентов первого игрока:

$$r_6 + r_8 = 0.133759 + 0.095231 = 0.228990.$$

Предельная влияние агента второго игрока:

$$r_9 = 0.087161.$$

Так как суммарная влияние агентов первого игрока выше, его управление сильнее повлияло на итоговое мнение.

Вывод:

В ходе лабораторной работы была рассмотрена модель информационного противоборства в социальной сети из 10 агентов.

Была сгенерирована стохастическая матрица доверия A . Проверка показала, что все элементы матрицы неотрицательны, а суммы строк равны 1, следовательно, матрица корректно описывает распределение доверия агентов.

При моделировании без информационного управления начальный вектор мнений

$$x(0) = (15; 17; 10; 3; 16; 13; 3; 9; 3; 16)$$

привёл к итоговому мнению

$$X_0 \approx 10.793976.$$

После введения информационного управления первый игрок воздействовал на агентов 6 и 8 значением $u = 34$, а второй игрок воздействовал на агента 9 значением $v = -44$. С учётом управления итоговое мнение социальной сети стало равно

$$X_1 \approx 11.887151.$$

Поскольку итоговое мнение оказалось ближе к цели первого игрока, чем к цели второго игрока, победителем информационного противоборства является первый игрок.

Таким образом, информационное управление действительно повлияло на итоговое мнение агентов, сместив его относительно результата без управления примерно на 1.093175.

Приложение (исходный код)

```
from __future__ import annotations

from dataclasses import dataclass
from typing import Iterable

import numpy as np

N_AGENTS = 10
SEED = 2026
EPS = 1e-6
MAX_ITER = 10_000

INITIAL_LOW = 1
INITIAL_HIGH = 20

PLAYER_1_LOW = 0
PLAYER_1_HIGH = 100
PLAYER_2_LOW = -100
PLAYER_2_HIGH = 0

@dataclass(frozen=True)
class SimulationResult:
    x0: np.ndarray
    final_x: np.ndarray
    iterations: int
    last_delta: float
    convergence_rows: list[tuple[int, float, float, float, float]]

@dataclass(frozen=True)
class InfluenceScenario:
    player_1_agents: list[int]
    player_2_agents: list[int]
    neutral_agents: list[int]
    u: int
    v: int
    x0: np.ndarray

def fmt_vector(values: Iterable[float], digits: int = 3) -> str:
    return "(" + "; ".join(f"{x:.{digits}f}" for x in values) + ")"

def fmt_int_vector(values: Iterable[int]) -> str:
    return "(" + "; ".join(str(int(x)) for x in values) + ")"

def print_matrix(name: str, matrix: np.ndarray, digits: int = 3) -> None:
    print(f"{name} =")
    for row in matrix:
        print(" " + " ".join(f"{x:8.{digits}f}" for x in row))
    print()

def to_agent_numbers(indices: Iterable[int]) -> list[int]:
    return [i + 1 for i in indices]

def generate_trust_matrix(n: int, rng: np.random.Generator) -> np.ndarray:
    raw = rng.random((n, n)) + 1e-3
    return raw / raw.sum(axis=1, keepdims=True)

def generate_initial_opinions(n: int, rng: np.random.Generator) -> np.ndarray:
    return rng.integers(INITIAL_LOW, INITIAL_HIGH + 1, size=n).astype(float)

def choose_influence_scenario(
    n: int,
    rng: np.random.Generator,
    base_x0: np.ndarray,
) -> InfluenceScenario:
    max_count = max(1, n // 3)
```

```

count_1 = int(rng.integers(1, max_count + 1))
count_2 = int(rng.integers(1, max_count + 1))

permutation = rng.permutation(n)
player_1_agents = sorted(map(int, permutation[:count_1]))
player_2_agents = sorted(map(int, permutation[count_1:count_1 + count_2]))
used = set(player_1_agents) | set(player_2_agents)
neutral_agents = [i for i in range(n) if i not in used]

u = int(rng.integers(PPLAYER_1_LOW, PPLAYER_1_HIGH + 1))
v = int(rng.integers(PPLAYER_2_LOW, PPLAYER_2_HIGH + 1))

x0 = base_x0.astype(float).copy()

x0[player_1_agents] = u
x0[player_2_agents] = v

return InfluenceScenario(
    player_1_agents=player_1_agents,
    player_2_agents=player_2_agents,
    neutral_agents=neutral_agents,
    u=u,
    v=v,
    x0=x0,
)

def is_row_stochastic(A: np.ndarray, tol: float = 1e-10) -> bool:
    non_negative = np.all(A >= -tol)
    row_sums_are_one = np.allclose(A.sum(axis=1), 1.0, atol=tol)
    return bool(non_negative and row_sums_are_one)

def simulate_opinions(A: np.ndarray, x0: np.ndarray, eps: float, max_iter: int) ->
SimulationResult:
    x_prev = x0.astype(float).copy()
    convergence_rows: list[tuple[int, float, float, float, float]] = []

    for t in range(1, max_iter + 1):
        x_next = A @ x_prev
        delta = float(np.max(np.abs(x_next - x_prev)))
        min_x = float(np.min(x_next))
        max_x = float(np.max(x_next))
        spread = max_x - min_x
        convergence_rows.append((t, delta, min_x, max_x, spread))

        if delta < eps:
            return SimulationResult(
                x0=x0,
                final_x=x_next,
                iterations=t,
                last_delta=delta,
                convergence_rows=convergence_rows,
            )
        x_prev = x_next

    raise RuntimeError(f"Мнения не сошлись за {max_iter} итераций. Последняя delta = {delta}")

def stationary_row(A: np.ndarray, eps: float, max_iter: int) -> tuple[np.ndarray, int, float]:
    n = A.shape[0]
    r_prev = np.full(n, 1.0 / n)

    for t in range(1, max_iter + 1):
        r_next = r_prev @ A
        delta = float(np.max(np.abs(r_next - r_prev)))
        if delta < eps:
            return r_next, t, delta
        r_prev = r_next

    raise RuntimeError("Предельная строка матрицы доверия не была найдена за max_iter итераций.")

def limiting_matrix_power(A: np.ndarray, eps: float, max_iter: int) -> tuple[np.ndarray, int,
float]:
    n = A.shape[0]

```

```

A_prev = np.eye(n)

for k in range(1, max_iter + 1):
    A_next = A_prev @ A
    delta = float(np.max(np.abs(A_next - A_prev)))

    if k > 1 and delta < eps:
        return A_next, k, delta

    A_prev = A_next

raise RuntimeError(
    f"Матрица A^k не сошлась за {max_iter} итераций. Последняя delta = {delta}"
)

def interpret_winner(final_value: float, player_1_target: float, player_2_target: float) -> str:
    distance_to_player_1 = abs(final_value - player_1_target)
    distance_to_player_2 = abs(final_value - player_2_target)

    if distance_to_player_1 < distance_to_player_2:
        return (
            "выиграл первый игрок\n"
            f"расстояние до цели первого игрока = {distance_to_player_1:.6f}\n"
            f"расстояние до цели второго игрока = {distance_to_player_2:.6f}\n"
        )

    if distance_to_player_2 < distance_to_player_1:
        return (
            "выиграл второй игрок\n"
            f"расстояние до цели первого игрока = {distance_to_player_1:.6f}\n"
            f"расстояние до цели второго игрока = {distance_to_player_2:.6f}\n"
        )

    return (
        "ничья: итоговое мнение равноудалено от целей игроков "
        f"расстояние = {distance_to_player_1:.6f}"
    )

def main() -> None:
    rng = np.random.default_rng(SEED)

    print("Лабораторная работа № 6")
    print("Информационное противоборство в социальных сетях")
    print("Закон изменения мнений:  $x(t) = A x(t-1)$ ")
    print(f"Количество агентов: {N_AGENTS}")
    print(f"Seed случайной генерации: {SEED}")
    print(f"Точность остановки eps: {EPS:g}")
    print("=" * 80)

    A = generate_trust_matrix(N_AGENTS, rng)
    print("1. Сгенерированная матрица доверия")
    print_matrix("A", A, digits=3)
    print("Проверка стохастичности матрицы доверия:")
    print(f"все элементы неотрицательны: {np.all(A >= 0)}")
    print(f"суммы строк: {fmt_vector(A.sum(axis=1), digits=6)}")
    print(f"матрица стохастическая по строкам: {is_row_stochastic(A)}")
    print("=" * 80)

    x0_plain = generate_initial_opinions(N_AGENTS, rng)
    plain_result = simulate_opinions(A, x0_plain, EPS, MAX_ITER)
    r, r_iters, r_delta = stationary_row(A, EPS, MAX_ITER)
    A_limit, A_limit_iters, A_limit_delta = limiting_matrix_power(A, EPS, MAX_ITER)
    theoretical_plain = float(r @ x0_plain)

    print("2. Моделирование без информационного управления")
    print(f"Начальные мнения x(0): {fmt_int_vector(x0_plain.astype(int))}")
    print_convergence_table(plain_result.convergence_rows)
    print(f"Результирующие мнения x({plain_result.iterations}): {fmt_vector(plain_result.final_x)}")
    print(f"Число итераций до сходимости: {plain_result.iterations}")
    print(f"Последнее max|x(t)-x(t-1)|: {plain_result.last_delta:.8f}")
    print(f"Предельная строка r матрицы A^k: {fmt_vector(r, digits=6)}")
    print(f"Полная матрица A^k при k = {A_limit_iters}:")
    print_matrix(f"A^{A_limit_iters}", A_limit, digits=6)

```

```

print(f"Последнее  $\max|A^k - A^{(k-1)}|$ : {A_limit_delta:.8f}")
print(f"Проверка по формуле  $X = r * x(0)$ : {theoretical_plain:.6f}")
print("=" * 80)

scenario = choose_influence_scenario(N_AGENTS, rng, x0_plain)
influence_result = simulate_opinions(A, scenario.x0, EPS, MAX_ITER)
theoretical_influence = float(r @ scenario.x0)
final_value = float(np.mean(influence_result.final_x))

print("3. Моделирование с информационным управлением")
print(f"Агенты влияния первого игрока: {to_agent_numbers(scenario.player_1_agents)}")
print(f"Агенты влияния второго игрока: {to_agent_numbers(scenario.player_2_agents)}")
print(f"Нейтральные агенты: {to_agent_numbers(scenario.neutral_agents)}")
print(f"Управление первого игрока u: {scenario.u}")
print(f"Управление второго игрока v: {scenario.v}")
print("Начальные мнения нейтральных агентов:")
for i in scenario.neutral_agents:
    print(f"    агент {i + 1}: {int(scenario.x0[i])}")
print(f"Начальный вектор с учетом управления x(0):")
{fmt_int_vector(scenario.x0.astype(int))}
print_convergence_table(influence_result.convergence_rows)
print(f"Результирующие мнения x({influence_result.iterations}):")
{fmt_vector(influence_result.final_x)}
print(f"Число итераций до сходимости: {influence_result.iterations}")
print(f"Последнее  $\max|x(t) - x(t-1)|$ : {influence_result.last_delta:.8f}")
print(f"Проверка по формуле  $X = r * x(0)$ : {theoretical_influence:.6f}")
print(f"Итоговое мнение: {final_value:.6f}")
print(f"{interpret_winner(final_value, scenario.u, scenario.v)}")
print("=" * 80)

def print_convergence_table(rows: list[tuple[int, float, float, float, float]]) -> None:
    print("Таблица сходимости:")
    print("    t |  $\max|x(t) - x(t-1)|$  | min x(t) | max x(t) | разброс")
    print("-----+-----+-----+-----+-----")
    for t, delta, min_x, max_x, spread in rows:
        print(f"{t:3d} | {delta:17.8f} | {min_x:8.3f} | {max_x:8.3f} | {spread:7.3f}")
    print()

if __name__ == "__main__":
    main()

```